CS3006 Parallel and Distributed Computing · Assignment 2

# Building a Parallel Task Execution Library

Implementation, measurements and analysis



| | |
|---|---|
| **Student** | Izen Abbas |
| **Roll number** | 23L-0748 |
| **Program / semester** | BS(CS)-7G · Fall 2026 |
| **Instructor** | Mubashar Hussain |
| **Due date** | Wednesday, 14 October 2026 |





1. Setup and measurement method
2. Design of the task systems
3. Part 1 — Parallel spawn
4. Part 2 — Spinning thread pool
5. Part 3 — Sleeping thread pool
6. Part 4 — Scheduling and granularity
7. Part 5 — Asynchronous task graphs
8. Combined performance analysis
9. Appendix A — Full $T_1$, $T_p$, $S_p$, $E_p$ table
10. Appendix B — Correctness evidence

# 1. Setup and measurement method

| | |
|---|---|
| **CPU** | Intel Core i7-10875H (Comet Lake), one socket |
| **Cores / hardware threads** | 8 physical cores, 16 hardware threads (2-way SMT) |
| **Clock** | Turbo Boost disabled ( `intel_pstate/no_turbo = 1` ), so every core runs at ≤ 2.3 GHz. This keeps run-to-run timing stable. |
| **Caches** | 32 KB L1d and 256 KB L2 per core; 16 MB shared L3 |
| **Operating system** | Ubuntu 26.04.1 LTS, Linux 7.0.0-38-generic |
| **Compiler** | g++ 15.2.0 with the supplied Makefile flags: `-O3 -std=c++11 -Wall` |

**Interpreting worker counts.** With p ≤ 8, every worker can have its own physical core. At p = 16, two workers share each core as SMT siblings. Siblings share the core's execution units, so the second one adds much less than a real core, and gains between p = 8 and p = 16 should be small. The calling thread is not counted in p. It sleeps in Spawn (in `join`) and in Sleep (on a condition variable), but it busy-waits in Spin. So Spin at p = 16 has 17 runnable threads on 16 hardware threads.

**Statistic.** Every reported time is the **minimum of 5 runs**. `./runtasks -n p -i 5 <test>` creates a fresh task system for each run and prints the minimum for each implementation. The minimum is used throughout.

**Definitions. $T_1$ is the runtime of the supplied serial implementation** ( `TaskSystemSerial` ): the minimum over all its runs in the sweep, which is 25 runs per workload. $S_p = T_1/T_p$ and $E_p = S_p/p$, where p is the number of worker threads ( `-n` ). Because $T_1$ is the serial code, $S_p$ includes every overhead a task system adds. That is why Spawn's $S_1$ can be below 1.

**What is timed.** The harness times only the test's `run()` / `sync()` calls. Creating and destroying a thread pool happens outside the timed region. Spawn creates its threads inside `run()`, so that cost is always timed. This is the correct comparison for a long-lived runtime, but it hides the one-off pool start-up cost.

**Sweep.** p ∈ {1, 2, 4, 8, 16} for all 11 supplied synchronous workloads and the Part 4 workloads ( `part1to4_synchronous` build), and for the Part 5 tests ( `part5_async_deps` build).

Supplied synchronous workloads (from `tests/tests.h` )

| Workload | Launches × tasks | One task | Character |
|---|---|---|---|
| `super_super_light` | 400 × 64 | copies 512 ints | almost no work per task |
| `super_light` | 400 × 64 | 512 elements × 32-iteration loop | light tasks |
| `ping_pong_equal` | 400 × 64 | 8192 elements, equal cost | streams 4 MB per launch |
| `ping_pong_unequal` | 400 × 64 | cost falls linearly with task id | non-uniform: task 0 ≈ 2× average, task 63 ≈ 0 |
| `recursive_fibonacci` | 30 × 256 | recursive fib(25) | coarse, compute-bound |
| `math_operations_in_tight_for_loop` | 2000 × 16 | 32 elements × 150 exp/log/mul | medium tasks, many launches |
| `…_fewer_tasks` | 2000 × 9 | 56–64 elements | only 9 tasks per launch |
| `…_fan_in` | 256 × 64, then 1 × 1 | math tasks, then one reduce | serial reduce at the end |
| `…_reduction_tree` | 32 × 64, then 31 × 1 | math tasks, then pairwise reduces | 31 single-task launches |
| `spin_between_run_calls` | 3 launches: 1, 2, 1 tasks | the 2 middle tasks compute fib(40) | at most 2 workers busy |
| `mandelbrot_chunked` | 1 × 128 | interleaved image rows | one coarse launch |

## 2. Design of the task systems

All three parallel task systems assign task ids **dynamically, in increasing order**: a free worker claims the next unclaimed id from a shared counter. A task is counted as **finished only after `runTask()` returns**, never when it is claimed. The calling thread does not execute tasks; it only waits.

### 2.1 Spawn (Part 1) — `TaskSystemParallelSpawn::run`

```
std::atomic<int> next_task(0);
auto worker = [&]() {
    for (int id = next_task++; id < num_total_tasks; id = next_task++)
        runnable->runTask(id, num_total_tasks);
};
// min(num_threads, num_total_tasks) threads run worker(); the caller joins all of them.
```

**Shared scheduler state.**

- **The next-task counter.** Every worker read-modify-writes it, so it is a `std::atomic<int>`. `fetch_add` returns each value exactly once, so every id in [0, n) goes to exactly one worker.
- **`runnable` and `num_total_tasks`.** Shared, but read-only during the launch. Creating a thread makes them visible to it.
- **Task outputs.** Written by the tasks and made visible to the caller by `join()`. That is also what makes `run()` synchronous: it returns only after every worker has run out of ids and exited.

No more than `num_threads` threads are ever created, and none is created that would find no task.

### 2.2 Spinning pool (Part 2) — `TaskSystemParallelThreadPoolSpinning`

The constructor starts `num_threads` workers that live until the destructor sets `shutdown_` and joins them. `mutex_` guards the current launch (`runnable_`, `num_total_tasks_`, `next_task_`). `tasks_done_`, `has_work_` and `shutdown_` are atomics.

```
while (!shutdown_) {                              // worker loop
    if (!has_work_) continue;                     // idle: spin on an atomic load
    std::unique_lock<std::mutex> lock(mutex_);
    if (next_task_ >= num_total_tasks_) continue;   // lost the race for the last task
    int task_id = next_task_++;
    if (next_task_ == num_total_tasks_) has_work_ = false;
    /* copy runnable_ and num_total_tasks_, unlock, run the task */
    tasks_done_++;                                // counted after the task finished
}
// run(): publish the launch under mutex_ (reset counters, has_work_ = n > 0),
//        then spin: while (tasks_done_ < n) {}
```

Idle workers spin on a plain atomic load (`has_work_`) and take the mutex only when a task is probably available. A spin built from repeated lock/unlock would make idle workers fight working ones for the mutex. The design stays correct across consecutive launches for two reasons:

1. Every increment of `tasks_done_` for launch k happens before `run(k)` returns, so the reset in `run(k+1)` cannot drop or misattribute an increment.
2. Claims happen under the same mutex that publishes a launch, so a worker can never pair a task id of launch k+1 with the runnable of launch k.

### 2.3 Sleeping pool (Part 3) — `TaskSystemParallelThreadPoolSleeping`

```
std::unique_lock<std::mutex> lock(mutex_);       // worker loop
while (true) {
    work_cv_.wait(lock, [this] { return shutdown_ || next_task_ < num_total_tasks_; });
    if (shutdown_) return;
    int task_id = next_task_++;
    /* copy runnable_ and num_total_tasks_ */
    lock.unlock();  runnable->runTask(task_id, num_total_tasks);  lock.lock();
    if (++tasks_done_ == num_total_tasks_) done_cv_.notify_one();   // last task wakes run()
}
// run(): lock; publish the launch; work_cv_.notify_all();
//        done_cv_.wait(lock, [this] { return tasks_done_ == num_total_tasks_; });
```

All state is guarded by one mutex. After finishing a task, a worker keeps the lock to record the completion and claim its next task, so each task costs one lock round trip. The destructor sets `shutdown_` under the lock, calls `notify_all()` and joins. No thread busy-waits anywhere.